Nama : Imron
NPM : 232310021
Matakuliah : Lab PW TI24PA2
Pertemuan 4

- Instalasi node.js

```
Microsoft Windows [Version 10.0.19045.6466]
(c) Microsoft Corporation. All rights reserved.

C:\Users\user>npm -v
11.11.0

C:\Users\user>node -v
v24.14.1

C:\Users\user>
```

- Membuat MERN Stack

```
Directory of C:\2026\PW\MERN-STACK

04/23/2026  10:23 AM    <DIR>          .
04/23/2026  10:23 AM    <DIR>          ..
04/23/2026  10:01 AM    <DIR>          backend
04/23/2026  10:24 AM    <DIR>          first-project-react
```

```
BACKEND
  node_modules
  index.js
  package-lock.json
  package.json
```

Didalam backend\index.js

1. Import dan inisialisasi

```
const express = require("express");
const mysql = require("mysql2");
const cors = require("cors");
const app = express();
```

   Pada bagian ini dilakukan proses import library yang dibutuhkan, yaitu Express untuk membuat server, MySQL untuk koneksi database, dan CORS untuk mengatur izin akses. Setelah itu, const app sebagai inisialisasi server menggunakan Express.

2. Konfigurasi CORS

```
app.use(
  cors({
    origin: "http://localhost:3000",
    credentials: true,
  }),
);
app.use(express.json());
```

   Bagian ini digunakan untuk mengatur server:
   - CORS dikonfigurasi agar hanya frontend dari localhost:3000 yang diizinkan mengakses server.
   - express.json() digunakan agar server dapat membaca data dengan format JSON yang dikirim dari client.

3. Koneksi ke database

```
const db = mysql.createConnection({
  host: "localhost",
  user: "root",
  password: "",
  database: "pw_praktikum_db",
});

db.connect((err) => {
  if (err) {
    console.error("Koneksi ke database gagal: ", err.message);
    process.exit(1);
  }
  console.log("Koneksi ke database MYSQL Berhasil");
});
```

Pada bagian ini dibuat koneksi ke database MySQL dengan konfigurasi yang telah ditentukan. Sistem kemudian melakukan pengecekan:

- Jika koneksi gagal, program akan menampilkan pesan error dan dihentikan.
- Jika berhasil, akan ditampilkan pesan bahwa koneksi berhasil.

4. Endpoint root

```
app.get("/", (req, res) => {
  res.send("Server Berjalan");
});
```

Endpoint ini berfungsi sebagai indikator bahwa server aktif. Ketika alamat utama diakses, server akan memberikan respon sederhana berupa teks.

5. Running server

```
app.listen(3001, () => {
  console.log("Server running on port 3001");
});
```

Kode ini digunakan untuk menjalankan server pada port 3001 sehingga dapat diakses oleh client atau frontend.

6. Endpoint API

```
app.get("/api/info", (req, res) => {
  res.json({
    message: "API MERN Stack Build By express js",
    version: "1.0.0",
    status: "active",
    timestamp: new Date(),
  });
});
```

Ini merupakan endpoint yang menyediakan data dalam format JSON. Ketika endpoint /api/info diakses, server akan mengirimkan informasi berupa pesan, versi, status, dan waktu saat ini.

Pada project react src\app\page.tsx

1. Import

```
"use client";

import axios from "axios";
import { useEffect, useState } from "react";
```

Bagian ini menunjukkan bahwa file dijalankan di sisi client (browser). Selanjutnya dilakukan import:

- axios untuk mengambil data dari API
- useState untuk menyimpan data
- useEffect untuk menjalankan proses saat halaman pertama kali dimuat

2. Inisialisasi komponen dan state

```
6  export default function Home() {
7    const [info, setInfo] = useState<any>(null);
```

Membuat komponen utama bernama Home. State info digunakan untuk menyimpan data yang diterima dari API, dengan nilai awal null.

3. Mengambil data API

```
9    const GetAPIInfo = async () => {
10     try {
11       const response = await axios.get("http://localhost:3001/api/info");
12       setInfo(response.data);
13     } catch (error) {
14       console.error("error fetching API data:", error);
15       setInfo(error?.message);    Property 'message' does not exist on typ
16     }
17   };
```

Fungsi ini digunakan untuk mengambil data dari backend:

- Melakukan request ke endpoint /api/info menggunakan axios
- Jika berhasil, data disimpan ke state info
- Jika terjadi error, pesan error akan ditampilkan dan juga disimpan ke state

4. useEffect

```
19   useEffect(() => {
20     GetAPIInfo();
21   }, []);
```

useEffect digunakan untuk menjalankan fungsi GetAPIInfo() saat komponen pertama kali ditampilkan. Array kosong [] menandakan bahwa fungsi hanya dijalankan satu kali saat awal load.

5. Render tampilan

```
23   return (
24     <div>
25       <h1>Hello World</h1>
26       {info && (
27         <pre
28           style={{
29             backgroundColor: "#f4f4f4",
30             padding: "15px",
31             borderRadius: "8px",
32             border: "1px solid #ddd",
33             overflow: "auto",
34           }}
35         >
36           {JSON.stringify(info, null, 2)}
37         </pre>
38       )}
39     </div>
40   );
41 }
```

Mengatur tampilan halaman:

- Menampilkan judul “Hello World”
- Jika data info sudah tersedia, maka akan ditampilkan dalam bentuk teks JSON
- JSON.stringify() digunakan agar data tampil rapi dan mudah dibaca

- Menjalankan server menggunakan fitur –watch bawaan node.js agar server otomatis re-start saat ada perubahan dan tidak perlu install nodemon

```
Restarting 'index'
Server running on port 3001
Koneksi ke database MYSQL Berhasil
```

- Menampilkan hasil API di frontend